

ВНУТРЕННИЙ ТЕХ-ДОКЛАД

Как устроена LLM и как её развернуть

Разбираем на двух открытых моделях: IBM Granite 4.1 3B как чистая классика и Qwen3.8-27B как современный гибрид — от карточки на Hugging Face до продового инференса

huggingface.co/ibm-granite/granite-4.1-3b

· huggingface.co/Qwen/Qwen3.8-27B

О чём поговорим

1

Основы

Цикл предсказания токена + два кирпича: SwiGLU и RoPE

2

Анатомия

Attention изнутри, компромисс $O(n^2)$, GQA, RoPE и позиции

3

Два примера

Granite 4.1 3B и Qwen3.8-27B: гибрид, KV-кэш, бенчмарки

4

Ресурсы и обучение

Гигабайты, квантизация, SFT/RL, chat template, tool calling

5

Развёртывание

Transformers, vLLM/SGLang, llama.cpp, Docker/K8s

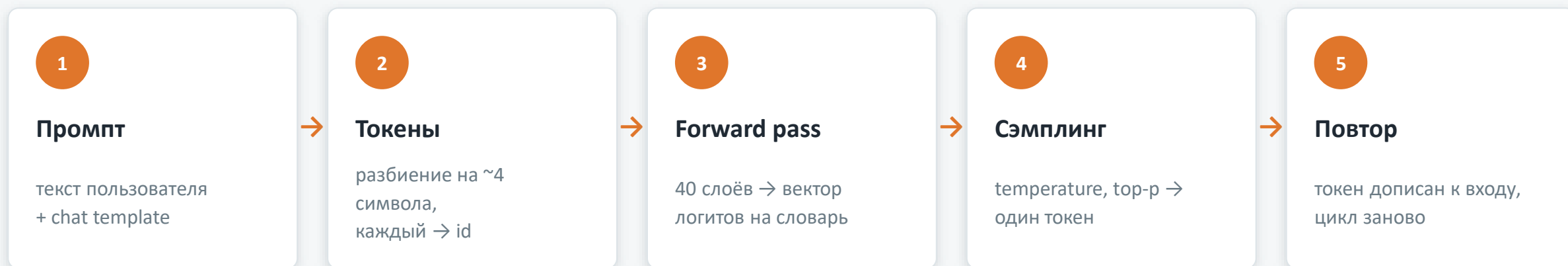
6

Как выбрать

Матрица решений + чек-лист для прода

LLM делает ровно одну вещь

Предсказывает распределение вероятностей следующего токена. Всё остальное — чат, агенты, RAG — надстройка над этим циклом.

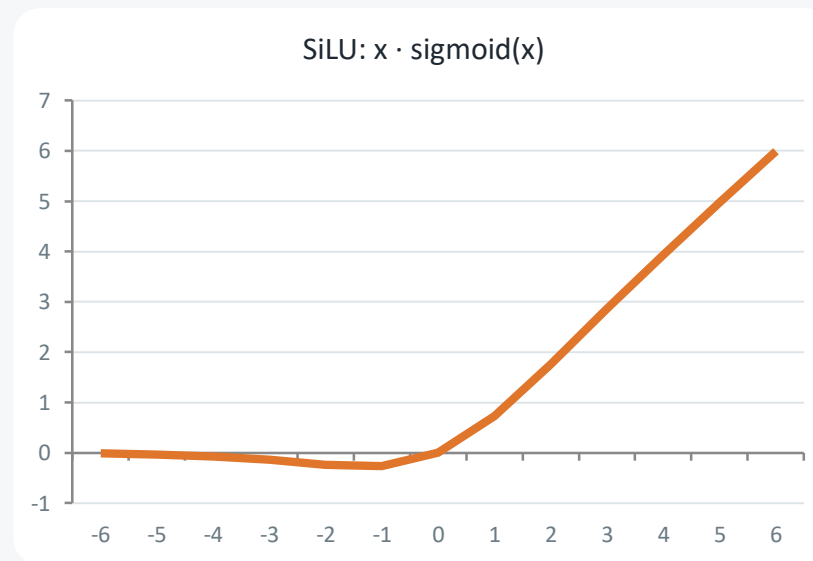


Практическое следствие

Генерация принципиально последовательна: 200 токенов ответа = 200 проходов по модели. Поэтому prefill (обработка промпта) параллелится и упирается в вычисления, а decode (генерация) — нет и упирается в пропускную способность памяти. Отсюда растут все трюки инференс-серверов.

SwiGLU: нелинейность с вентилем

Это то, из чего состоит примерно три четверти всех весов модели. Идея простая: одна ветка считает содержание, вторая решает, сколько его пропустить дальше.



Аналогия

Кран и труба. По трубе идёт вода (содержание), кран (вентиль) решает, сколько пропустить. Причём для каждого из 8192 каналов свой отдельный кран, и открывается он по-разному в зависимости от входа.

Почему не просто ReLU

SiLU гладкая и не обнуляет отрицательные значения полностью, а вентиль даёт модели управлять пропусканьем. Стабильно лучше — ценой третьей матрицы, из-за которой берут 8192, а не 10240.

RoPE: позиция как угол поворота

Модели нужно как-то отличать первое слово от пятого. RoPE не дописывает номер к вектору, а поворачивает его — тем сильнее, чем дальше токен стоит.

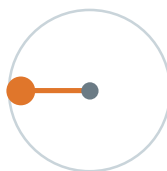
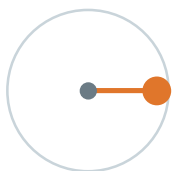
позиция 0

позиция 1

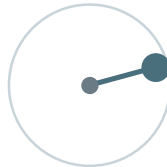
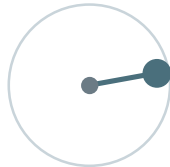
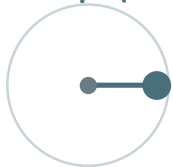
позиция 2

позиция 3

Быстрая пара (низкий индекс i) — большой шаг угла



Медленная пара (высокий индекс i) — маленький шаг



Вектор режется на пары чисел, каждая пара — своя стрелка. Быстрые различают соседей, медленные — далёкие части текста.

Аналогия

Часы со стрелками разной скорости. Секундная точно скажет, что было раньше — эта секунда или соседняя. Часовая — в какой части суток мы вообще находимся.

Что это даёт

Когда два повернутых вектора сравнивают между собой, результат зависит только от разницы углов — то есть от расстояния между токенами, а не от их абсолютных номеров.

Пока достаточно этой интуиции. Формулы, частоты и растяжение контекста разберём во второй трети доклада — там же станет понятно, почему именно поворот, а не сложение, и почему RoPE применяют не ко всему вектору сразу.

Паспорт нашего примера

3B

параметров

40

слоёв

2560

размер эмбединга

131 072

макс. длина контекста

Архитектура	decoder-only dense transformer
Внимание	GQA — 40 голов запросов, 8 KV-голов, размер головы 64
Позиции	RoPE (rotary position embeddings)
MLP	SwiGLU, скрытый размер 8192
Нормализация	RMSNorm; input/output эмбединги общие (tied)
Формат весов	BF16 · safetensors · лицензия Apache 2.0
Языки	12, включая английский, немецкий, испанский, японский, китайский

Модель целиком: 40 одинаковых слоёв

- RMSNorm → полный attention → прибавили к residual
- RMSNorm → MLP со SwiGLU → прибавили к residual

Оранжевый здесь и дальше означает «полный attention». У Granite других слоёв нет — значит, квадрат по длине платят все сорок, и KV-кэш копится тоже во всех сорока. Отсюда 40 в формуле кэша.

Стопка одинаковых блоков — это и есть вся архитектура. Дальше разбираем, что происходит внутри одного из них.

Что внутри одного из 40 слоёв

Слой не «понимает» текст. Он делает две вещи: смешивает информацию между токенами (attention) и обрабатывает каждый токен отдельно (MLP).



RMSNorm

нормализация входа, дешевле LayerNorm — без вычитания среднего



Grouped-Query Attention

40 query-голов на 8 KV-голов + RoPE на Q и K



+ residual

выход прибавляется ко входу — градиент течёт напрямую



RMSNorm

вторая нормализация



MLP co SwiGLU

2560 → 8192 → 2560, нелинейность и «память фактов»



+ residual

и так 40 раз подряд

Attention — «кто на кого смотрит»

- Каждый токен формирует Query, Key, Value
- $Q \cdot K^T \rightarrow$ веса внимания $\rightarrow$ взвешенная сумма V
- Causal mask: смотреть можно только назад
- Стоимость растёт квадратично по длине последовательности

MLP — где живут знания

- ~2/3 всех параметров модели — здесь
- SwiGLU: две проекции, одна как «вентиль»
- Работает с каждым токеном независимо
- Основной объём весов, которые мы квантизуем

Attention: как токены смотрят друг на друга

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}} + M\right) \cdot V$$

1

Три проекции

Из вектора токена линейно получаем Q («что я ищу»), K («что я предлагаю») и V («что я отдаю»). У Granite 2560 измерений режутся на 40 голов по 64.

2

Оценки схожести

$Q \cdot K^T$ даёт матрицу $n \times n$: насколько каждый токен релевантен каждому. Делим на $\sqrt{64}$, иначе softmax насыщается и градиенты умирают.

3

Причинная маска M

Позициям «в будущем» ставим минус бесконечность. Именно маска делает модель авторегрессионной и позволяет учить все позиции одним проходом.

4

Смешивание

Softmax по строке превращает оценки в веса, взвешенная сумма V становится новым вектором токена. Далее конкатенация голов и выходная проекция.

Почему это вообще работает

- Маршрут информации не зашит в веса, а вычисляется из самого содержимого — модель решает, что с чем связывать, прямо на инференсе
- Путь между любыми двумя токенами — один шаг, без затухания как в рекуррентных сетях
- Разные головы специализируются: порядок слов, кореференция, повтор n -грамм, разметка структуры
- Обучение полностью параллельно по позициям — это и сделало масштабирование возможным

Главный компромисс attention

Полная связность любой ценой: каждый token может достать любой другой за один шаг — и ровно за это мы платим квадратом по длине.

$O(n^2)$

Prefill упирается в вычисления

Матрица оценок растёт квадратично. 1k токенов — это 1 млн пар, 131k — уже 17 млрд пар, и так на каждой из 40 голов каждого из 40 слоёв.

$O(n)$

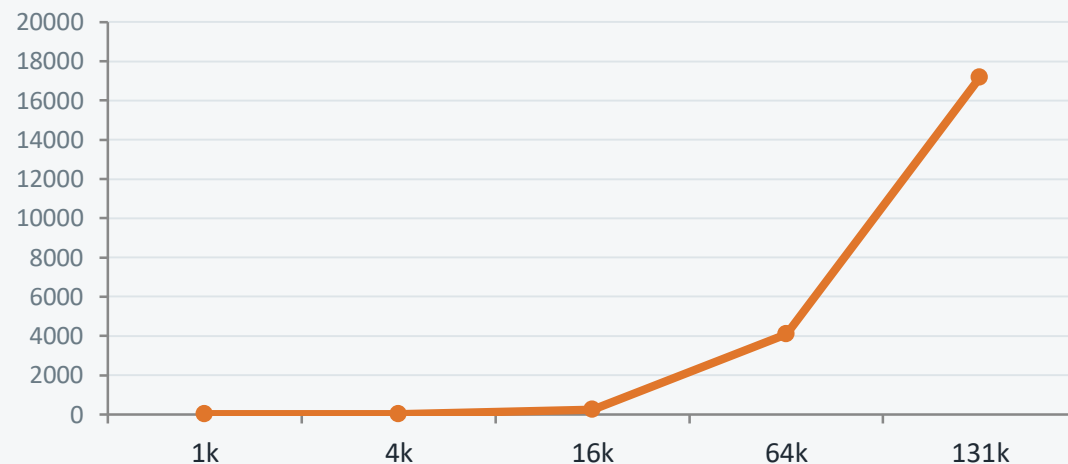
Decode упирается в память

На каждый новый token нужно прочитать весь KV-кэш целиком. Арифметическая интенсивность близка к нулю: GPU считает быстрее, чем память успевает отдавать.

Куда уходит вся инженерия вокруг LLM

Уменьшить n (окно внимания, RAG) · уменьшить KV на token (GQA, MQA, квантизация кэша) · не хранить лишнего (PagedAttention, prefix caching). FlashAttention убирает матрицу $n \times n$ из памяти.

Стоимость prefill относительно 1k токенов



Что мы получаем взамен

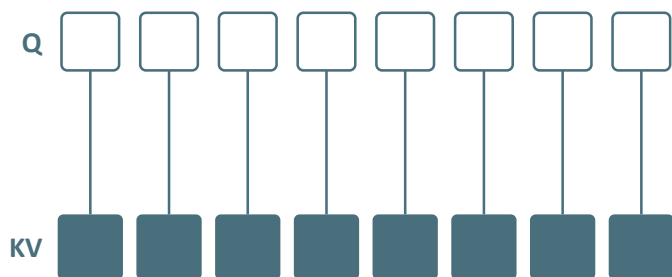
Ни одна более дешёвая архитектура пока не даёт такого же качества на длинных зависимостях. Квадрат — это цена за то, что модель ничего не забывает внутри окна.

Наглядно: MHA, GQA и MQA

Сверху — query-головы, снизу — наборы K и V. Линия означает «эта голова читает этот набор». Меняется только нижний ряд.

MHA

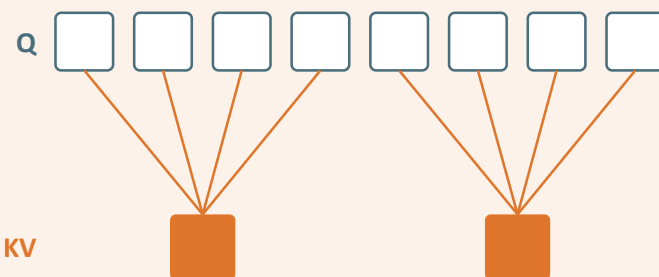
8 query · 8 KV



Кэш максимальный. Каждая голова тащит свой набор.

GQA

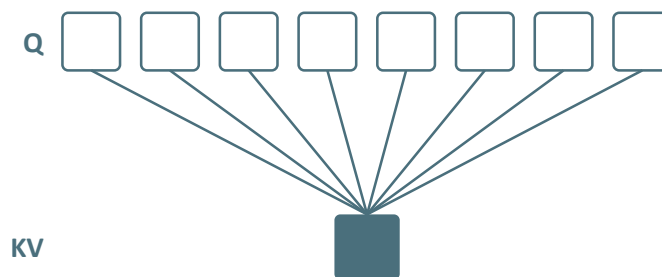
8 query · 2 KV



Группы по четыре делят один набор. Кэш в 4 раза меньше.

MQA

8 query · 1 KV



Один набор на всех. Дёшево, но качество проседает.

Число query-голов не меняется — меняется только то, сколько независимых наборов K и V приходится хранить в кэше.

Варианты attention: где именно экономят

Все подходы двигаются по одной шкале: меньше памяти и вычислений — больше риска потерять качество.

Вариант	KV на токен	Качество	Кто использует и в чём подвох
MHA — классика	1× (эталон)	Эталон	GPT-2, ранние модели. Кэш раздувается быстрее всего
MQA	1 / число голов	Заметная просадка	Одна KV-голова на все. Экономия максимальна, качество страдает
GQA	1 / размер группы	Почти без потерь	Llama 3, Qwen, Granite 4.1. Компромисс, ставший дефолтом
MLA	сжатие в латент	Малая просадка	DeepSeek. Сложнее в реализации и в поддержке движками
Sliding window	окно фиксировано	Теряет дальнее	Mistral, Gemma. Дальний контекст доступен только косвенно
Linear / SSM	O(n) без кэша	Другой профиль ошибок	Mamba и гибриды. Дёшево, но хуже точное извлечение из контекста

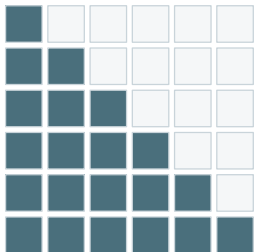
Бесплатных вариантов в этой таблице нет. GQA стал стандартом просто потому, что это самая дешёвая точка, где просадка качества ещё теряется в шуме замеров.

Линейное внимание: attention как RNN

Если заменить экспоненту на разложимое ядро, q выносится за знак суммы — и вся история сворачивается в одну матрицу состояния.

```
softmax:   o_i = SUM_j exp(q_i · k_j) v_j / SUM_j exp(q_i · k_j)    # exp не разложить -> нужен весь ряд
линейное:  exp(q · k) -> phi(q) · phi(k)    =>   S_i = S_{i-1} + phi(k_i) · v_i^T,   o_i = phi(q_i) · S_i
```

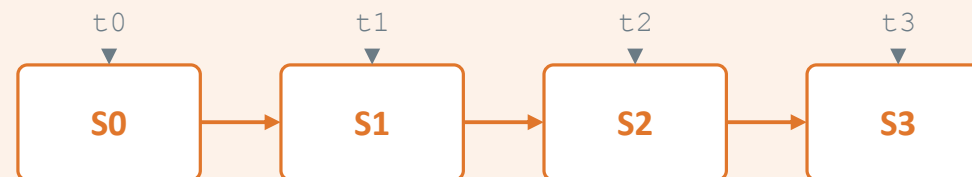
Полный attention — хранит всё



$n \times n$ оценок,
ниже диагонали —
видимое прошлое

Память на запрос растёт с длиной: KV-кэш линейно, матрица оценок квадратично.

Линейное — сворачивает в состояние



Матрица $d \times d$, размер не зависит от длины. KV-кэша нет вообще, стоимость по длине линейная.

Это буквально RNN: состояние фиксированного размера плюс правило обновления. Отсюда название работы, с которой всё началось, — «Transformers are RNNs».

Почему наивная версия не работала

Две поломки

- Состояние только накапливает: каждый token подмешивается, ничего не вытесняется — к середине текста всё размазано
- Softmax умеет быть острым и выбрать один token, простое ядро так не может

Два лекарства

Затухание: $S_i = \lambda \cdot S_{i-1} + k_i v_i^T$

Старое выцветает. В GLA коэффициент обучаемый и зависит от входа.

Delta: $S_i = S_{i-1} (I - b k k^T) + b k v^T$

Не «дописать», а перезаписать по ключу. Отсюда DeltaNet.

S4

структурированная A,
обучение через FFT



Mamba

селективность: B, C и шаг
зависят от входа



Mamba-2

доказали эквивалентность
линейному attention



Гибриды

3 линейных слоя
на 1 полный

Чем всё равно платят — и почему появились гибриды

Состояние фиксированного размера — это информационное бутылочное горлышко. Пересказать длинный документ линейная модель может, а найти в нём конкретный идентификатор или скопировать строку кода дословно — заметно хуже. Ключевое наблюдение: чтобы вернуть точное извлечение, хватает небольшой доли слоёв полного внимания — не половины, а одного из четырёх. Отсюда схема 3:1 у Qwen3.8.

GQA: почему 8 KV-голов, а не 40

Во время генерации K и V всех прошлых токенов лежат в памяти — это KV-кэш. Grouped-Query Attention заставляет группы query-голов делить один набор KV.

Классический МНА

40 query-голов · 40 KV-голов

400 КБ

на 1 токен контекста

53.7 ГБ

на полные 131k токенов

Каждая голова тащит свой кэш. На длинном контексте кэш перевешивает сами веса модели в разы.

GQA в Granite 4.1

40 query-голов · 8 KV-голов

80 КБ

на 1 токен контекста

10.7 ГБ

на полные 131k токенов

В 5 раз меньше памяти и в 5 раз меньше чтений из VRAM на каждом шаге декодирования — при почти неизменном качестве.

Формула: $2 \text{ (K и V)} \times \text{слои} \times \text{KV-головы} \times \text{размер головы} \times \text{байт на число} = 2 \times 40 \times 8 \times 64 \times 2 \approx 80 \text{ КБ на токен}$

Attention не видит порядок слов

Без позиционной информации attention перестановочно-эквивариантен: он работает с множеством токенов, а не с последовательностью.

Наглядно

```
"собака укусила человека"  
"человека укусила собака"
```

Одно и то же множество токенов. Чистый attention вернёт для них одинаковые представления — смысл различить нечем.

Два способа починить

- Подмешать позицию в сам вектор токена — так делали абсолютные эмбединги
- Вмешаться в вычисление оценок $Q \cdot K^T$ — так работают относительные схемы и RoPE, который выбрал Granite

Как менялись подходы

- **Обучаемые абсолютные**
BERT, GPT-2 — таблица векторов на позицию. За пределами обученной длины не работает вообще
- **Синусоидальные**
Оригинальный трансформер — фиксированные функции, экстраполяция посредственная
- **Относительный bias**
T5 — добавка к логиту по расстоянию. Работает, но дорого и плохо дружит с оптимизированными ядрами
- **RoPE**
Поворот Q и K. Относительность бесплатно, ноль параметров — сегодняшний стандарт
- **ALiBi и NoPE**
Линейный штраф по расстоянию или отказ от позиций вовсе — живые альтернативы, но нишевые

RoPE: поворот вместо сложения

Позиция не добавляется к вектору, а поворачивает его. Тогда скалярное произведение Q и K само начинает зависеть от расстояния между токенами.

Что происходит с вектором

- Вектор головы из 64 чисел разбивается на 32 пары — каждая пара живёт на своей плоскости
- Пара с индексом i поворачивается на угол $m \cdot \text{theta}_i$, где m — номер позиции в последовательности
- Низкие индексы вращаются быстро и кодируют локальный порядок, высокие — медленно и держат глобальную позицию

```
theta_i = base^(-2i/64),    base = 10000
```

```
<R_m q, R_n k> = f(q, k, m - n)
```

Ключевое свойство

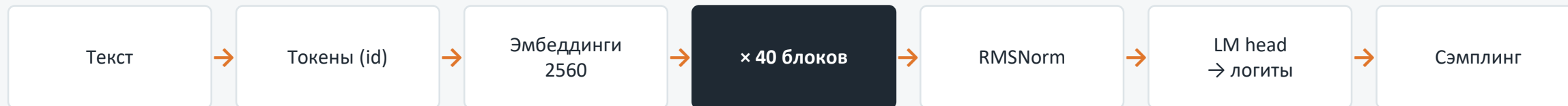
На входе мы подставляем абсолютные позиции m и n , а в оценке внимания остаётся только их разность. Абсолютное кодирование даёт относительное поведение — без единого дополнительного параметра.

Что важно на практике

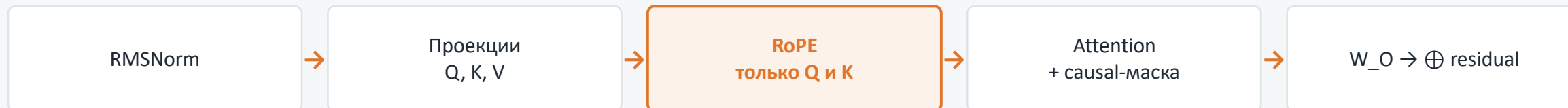
- Применяется к Q и K, но не к V — поэтому не мешает кэшированию значений
- Не добавляет обучаемых весов и почти ничего не стоит по времени
- База `rope_theta` — та самая ручка, которой растягивают контекст; она лежит в `config.json` модели

Полный путь токена и где включается RoPE

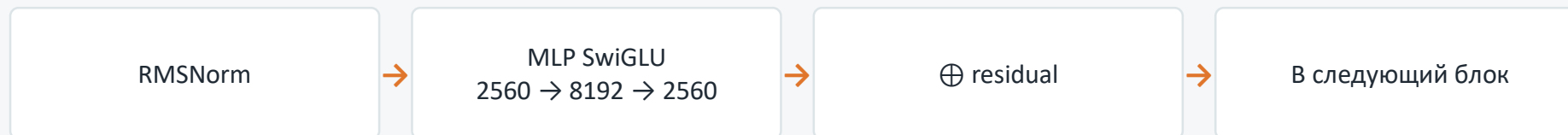
Сверху — вся модель целиком, снизу — что происходит внутри одного из 40 блоков. Оранжевым отмечено единственное место, где появляются позиции.



Шаг 1 — внимание



Шаг 2 — MLP



RoPE применяется заново в каждом из 40 блоков, только к Q и K и никогда к V — поэтому V-кэш остаётся позиционно-нейтральным. Больше нигде в модели позиции не появляются: ни отдельной таблицы, ни обучаемых параметров под них нет.

Как получают 131k и чем за это платят

Модель редко учат сразу на полной длине. Обычно берут более короткое обучение и растягивают позиции — а затем немного дообучают.

1

Position Interpolation

Сжимаем номера позиций в уже обученный диапазон. Просто и дёшево, но размывает локальное разрешение — соседние токены становятся неразличимее.

2

Смена базы (NTK-aware)

Увеличиваем `rope_theta`: медленные частоты растягиваются сильнее быстрых. Локальный порядок сохраняется лучше, чем при прямой интерполяции.

3

YaRN

Разные частоты растягиваются по-разному, плюс поправка температуры внимания. Сегодня фактический стандарт; обычно с коротким дообучением.

Заявленные 131 072 токена — это потолок, а не рекомендация

!

Память

10.7 ГБ KV-кэша на один запрос с полным окном — больше, чем сами веса (6 ГБ)

!

Скорость

Prefill квадратичен: 131k токенов промпта — это заметные секунды до первого токена

!

Качество

«Потеря середины»: факт в центре длинного окна находится хуже, чем в начале или в конце

Второй пример: Qwen3.8-27B

Granite показывает классическую схему в чистом виде. Qwen3.8-27B — та же основа, но со всеми современными трюками сразу. Удобно посмотреть, что за это время изменилось.

27B

параметров, dense

64

слоя

262 144

нативный контекст

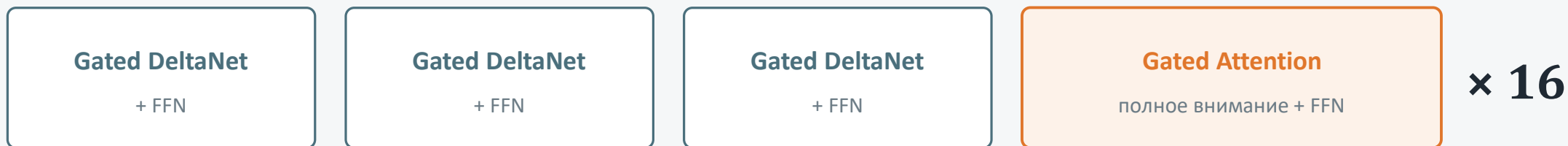
248 320

размер словаря

Тип	мультимодальная: текст, изображения, видео — causal LM плюс визуальный энкодер
Размеры	скрытый размер 5120, промежуточный размер FFN 17 408
Схема слоёв	16 × (3 × Gated DeltaNet + 1 × Gated Attention) — гибрид, а не чистый трансформер
Полное внимание	24 Q-головы на 4 KV-головы, размер головы 256, RoPE только на 64 измерениях
Контекст	262 144 нативно, до 1 000 000 через YaRN (rope_theta 10 млн, factor 4.0)
Рассуждения	thinking включён по умолчанию; reasoning_effort — low / medium / xhigh
Прочее	MTP (multi-token prediction) · BF16 · Apache 2.0 · 4.7 млн загрузок за месяц

Гибрид: три линейных слоя на один полный

Помните спектр вариантов attention? Qwen3.8 не выбирает одну точку на этой шкале, а чередует две — и повторяет связку 16 раз.



Gated DeltaNet — 48 слоёв из 64

Линейное внимание с состоянием фиксированного размера: 48 голов для V, 16 для QK, размер головы 128. Стоимость по длине линейная, кэш вообще не растёт. Хорошо смешивает информацию, но хуже находит конкретный факт в длинном тексте.

Gated Attention — 16 слоёв из 64

То самое полное внимание из первой части: 24 Q-головы на 4 KV, размер головы 256. Именно эти слои отвечают за точное извлечение из контекста — и только они платят квадрат по длине.

Зачем так

48 слоёв из 64 не платят квадрат по длине и не растят KV-кэш вообще. Модель получает качество, близкое к полному attention, при стоимости, близкой к линейной. Это прямой ответ на компромисс со слайда «Главный компромисс»: вместо выбора одной точки на шкале — смесь двух.

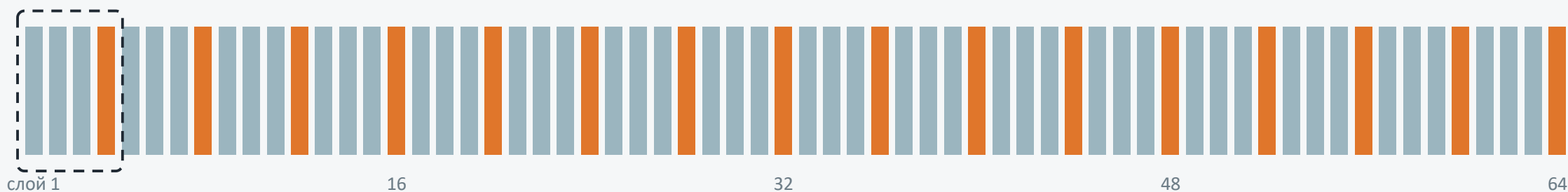
СТОПКА

64 слоя Qwen: каждый четвёртый — полный

Та же стопка, но слои разные. Три дешёвых линейных, один дорогой полный — и эта четвёрка повторяется шестнадцать раз подряд.

группа из 4 слоёв

... и так ещё 15 раз → всего 64 слоя



 Gated DeltaNet — линейный слой, 48 штук

 Gated Attention — полный attention, 16 штук

48 серых слоёв

Стоимость линейная по длине, состояние фиксированного размера. Кэша не растят вообще — сколько бы токенов ни пришло.

16 оранжевых слоёв

Единственные, кто платит квадрат по длине и копит KV-кэш. Именно они дают точное извлечение из контекста.

Ключ к чтению схемы: механизм перемешивания выбирается для каждого слоя отдельно, и он не обязан быть одинаковым во всей стопке.

Две стопки рядом

Один и тот же масштаб. Считаем не все слои, а только оранжевые — именно они попадают в формулу KV-кэша.

Granite 4.1 3B — 40 слоёв



40 слоёв
с полным attention

$$2 \times 40 \times 8 \text{ голов} \times 64 \times 2 \text{ байта} = 80 \text{ КиБ на токен}$$

Qwen3.8-27B — 64 слоя



16 слоёв
с полным attention

$$2 \times 16 \times 4 \text{ головы} \times 256 \times 2 \text{ байта} = 64 \text{ КиБ на токен}$$

Почему модель в девять раз больше стоит по контексту дешевле

В верхней стопке оранжевых сорок, в нижней — шестнадцать. Число параметров в формулу кэша не входит вообще: туда входят только слои с полным вниманием, число KV-голов и размер головы. Большой размер головы у Qwen (256 против 64) частично съедает выигрыш, но вчетверо меньшее число таких слоёв и вдвое меньшее число KV-голов перевешивают.

27B с кэшем меньше, чем у 3B

Размер KV-кэша определяется не числом параметров, а тремя множителями: сколько слоёв с полным вниманием, сколько KV-голов и какого они размера.

1.5 МБ

Гипотетический MHA 27B

$2 \times 64 \text{ слоя} \times 24 \text{ головы} \times 256$
байт на токен

80 КБ

Granite 4.1 3B, GQA

$2 \times 40 \text{ слоёв} \times 8 \text{ голов} \times 64$
байт на токен

64 КБ

Qwen3.8-27B, гибрид

$2 \times 16 \text{ слоёв} \times 4 \text{ головы} \times 256$
байт на токен

Что это даёт

Полные 262k контекста стоят около 16.8 ГБ кэша — сопоставимо с Granite 3B на его 131k. Модель в девять раз больше, а платить за длину приходится меньше.

Позиции: частичный RoPE и YaRN

Поворачиваются только 64 измерения головы из 256 — `partial_rotary_factor 0.25`. Растяжение до 1M делается YaRN с `rope_theta 10 млн`: ровно тот механизм, который мы разбирали двумя слайдами раньше.

Но веса никуда не делись: 27B в BF16 — это около 56 ГБ. На одну карту с 80 ГБ модель встаёт с запасом только в FP8, и именно в этом виде её обычно и раздают.

Считаем гигабайты честно

Три слагаемых на GPU: веса + KV-кэш + рабочие буферы. Забывают обычно про второе.

6.0 ГБ

Веса в BF16

3B параметров × 2 байта. Фиксированная величина.

+ N × 80 КБ

KV-кэш

Растёт линейно: длина × число одновременных запросов.

≈ 1–2 ГБ

Активации и буферы

CUDA-граф, промежуточные тензоры, фрагментация.

Сценарии на практике

Один пользователь, контекст 8k	≈ 6.6 ГБ	влезает в RTX 3060 12 ГБ
16 параллельных запросов по 8k	≈ 16 ГБ	нужна карта 24 ГБ (L4, A10, 3090)
Один запрос на все 131k	≈ 17 ГБ	длинный контекст дороже батча

Оценки для BF16 без квантизации; реальные цифры зависят от inference-движка и настроек предвыделения памяти

Квантизация: как ужать в 3–4 раза

Веса хранятся в меньшей разрядности. Качество проседает не линейно: до 4 бит потери обычно малы, ниже — быстро становятся заметными.

Формат	Размер весов	Где применяют	Комментарий
BF16 (оригинал)	≈ 6.0 ГБ	обучение, эталон	То, что лежит в репозитории safetensors
FP8 / INT8	≈ 3.0 ГБ	vLLM, SGLang на GPU	Почти без потерь, ускоряет decode
GGUF Q5_K_M	≈ 2.3 ГБ	llama.cpp, Ollama	Хороший баланс для CPU и Apple Silicon
GGUF Q4_K_M	≈ 2.0 ГБ	ноутбуки, edge	Рабочий дефолт для локального запуска
Q3 и ниже	< 1.7 ГБ	экзотика	Деградация уже видна на рассуждениях и коде

Для Granite 4.1 3B комьюнити уже выложило десятки готовых квантизаций — свою делать не нужно, достаточно взять GGUF из репозиториев, помеченных как quantized от этой базовой модели.

Как из архитектуры получается ассистент

1 Pretrain

Base-модель

Предсказание следующего токена на огромном корпусе. Здесь появляются знания и языки — но не умение отвечать на вопросы.



2 SFT

Инструкции

Дообучение на парах «запрос — ответ»: открытые датасеты с разрешительной лицензией, синтетика и отобранные людьми данные.



3 RL

Выравнивание

Циклы обучения с подкреплением. У Granite 4.1 это дало прирост именно в tool calling, следовании инструкциям и качестве диалога.

На чём это считали

Кластер NVIDIA GB200 NVL72 в CoreWeave: 72 GPU в одном NVLink-доме внутри стойки, между стойками — неблокирующий Fat-Tree InfiniBand на 400 Гбит/с. Тысячи GPU на одно обучение.

Что нам с этого

- Обучать с нуля мы не будем — берём готовое
- Наш уровень вмешательства: промпы, RAG, LoRA-адаптеры, реже — полный файнтюн
- Apache 2.0 разрешает коммерческое использование и дообучение

Chat template: где заканчивается модель

Модель не знает про «роли». Она видит одну плоскую строку со спецтокенами. Роли — это соглашение, зашитое в шаблон токенизатора.

Что пишет разработчик

```
chat = [  
    {"role": "user",  
     "content": "Назови одну лабораторию IBM в США"}  
]  
tokenizer.apply_chat_template(chat,  
    add_generation_prompt=True)
```

Что реально уходит в модель

```
<|start_of_role|>user<|end_of_role|>  
Назови одну лабораторию IBM в США  
<|end_of_text|>  
<|start_of_role|>assistant<|end_of_role|>  
# ← отсюда модель начинает генерацию
```

Три вывода для практики

Шаблон нельзя выдумывать

У каждой модели свои спецтокены. Взяли шаблон от Llama для Granite — получили тихую деградацию качества без единой ошибки в логах.

Токены реально расходуются

Служебная разметка и системный промпт съедают контекст и деньги на каждом запросе.

Инференс-серверы делают это за вас

vLLM и Ollama применяют шаблон из репозитория сами — поэтому у них OpenAI-совместимый `/chat/completions`.

Tool calling — это тоже просто текст

Никакой магии: описания функций подставляются в системный промпт, а «вызов» — это структурированный кусок текста, который модель научили генерировать.

1

Описываем функции

JSON-схема функций передаётся в `apply_chat_template` как `tools`

2

Модель решает

Возвращает блок `<tool_call>` с именем функции и аргументами в JSON

3

Выполняем мы

Модель ничего не вызывает сама. Парсим, дёргаем свой код, проверяем права

4

Возвращаем результат

Кладём ответ функции обратно в диалог и просим финальный ответ

Как выглядит ответ модели

```
<tool_call>
{"name": "get_current_weather",
 "arguments": {"city": "Boston"}}
</tool_call>
```

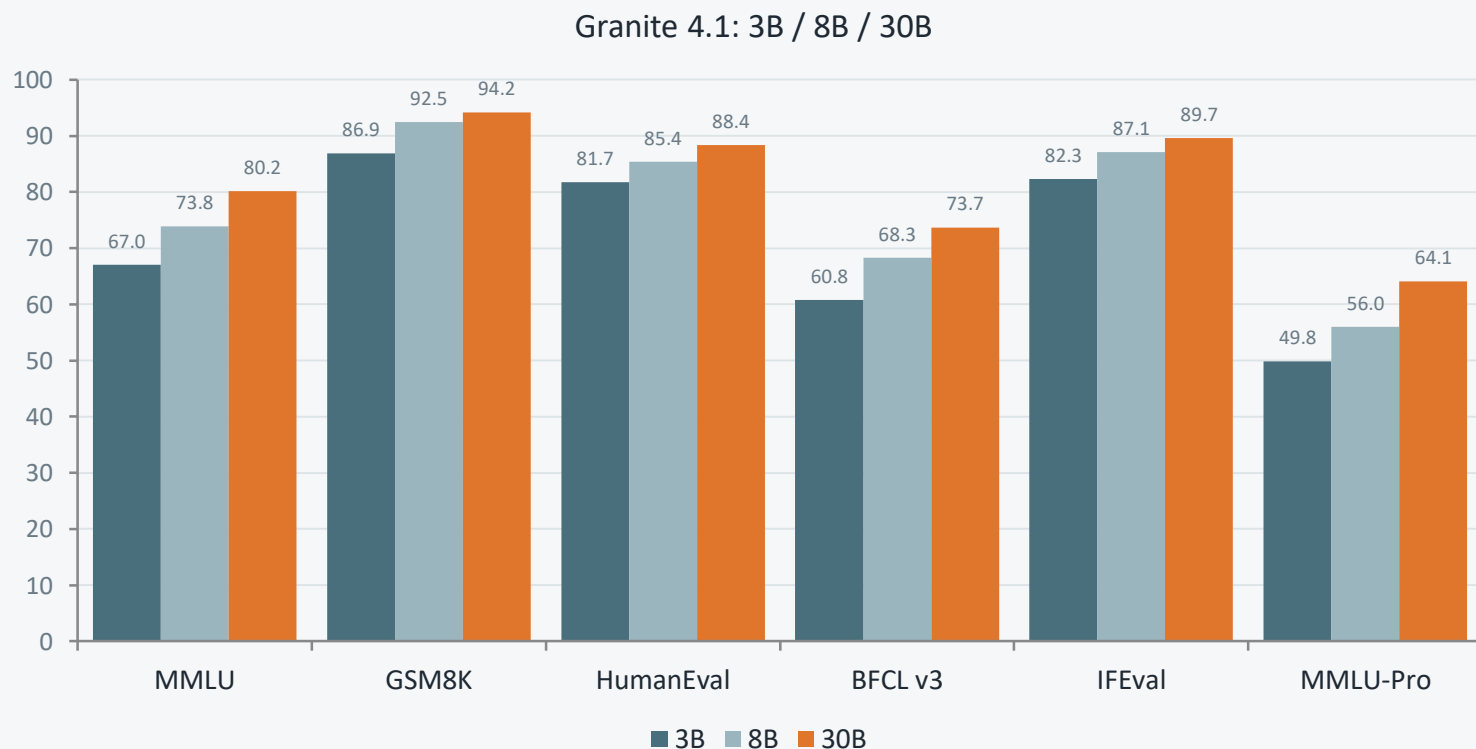
Метрика, на которую смотреть

BFCL v3 — 60.8

У 8B — 68.3, у 30B — 73.7. Если продукт агентный, разница между 3B и 8B здесь важнее, чем в общих знаниях.

Что даёт увеличение размера

Одно семейство, одинаковое обучение, разное число параметров. Видно, где размер решает, а где почти нет.

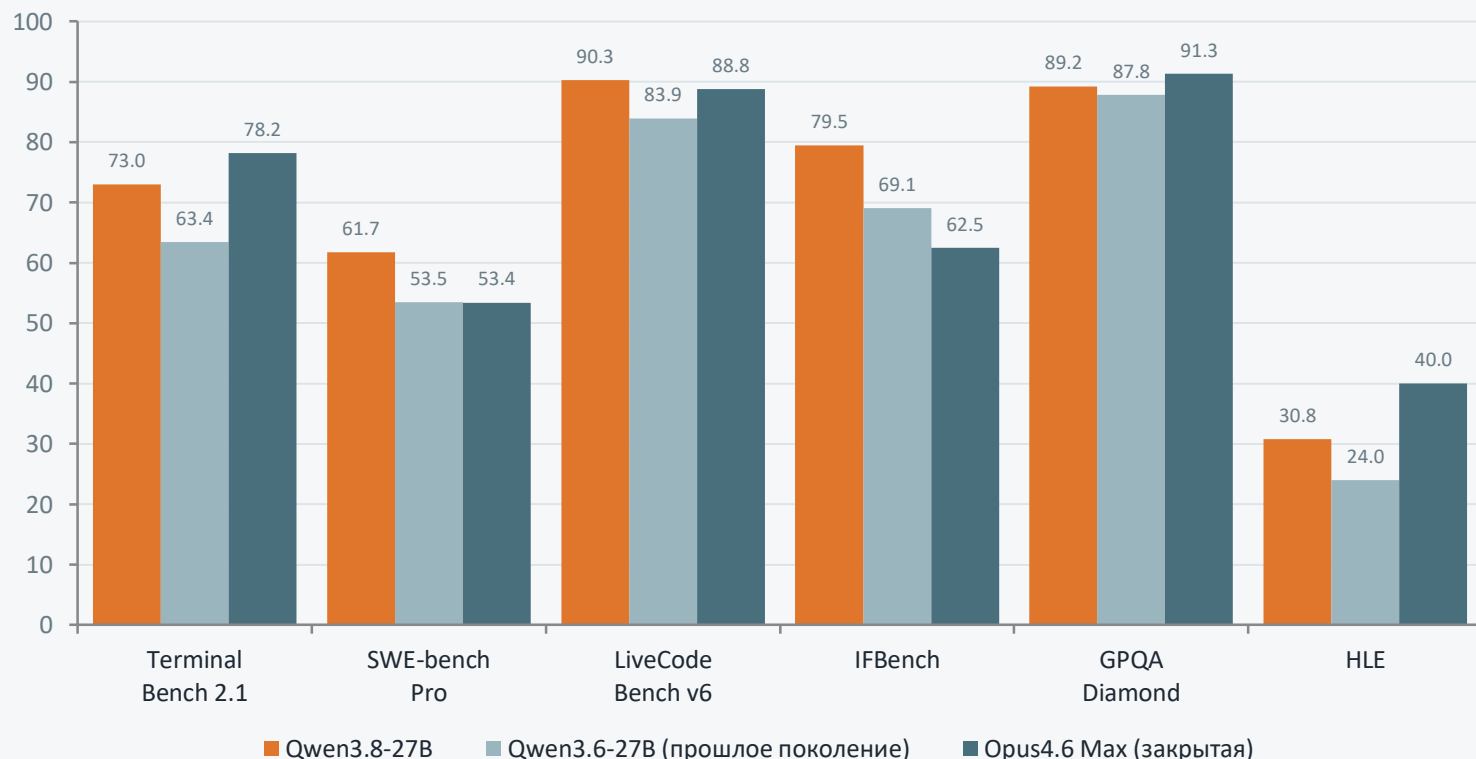


Как это читать

- Простая математика и базовый код почти насыщены уже на 3B — разрыв в единицы пунктов
- Сложные рассуждения (MMLU-Pro) и tool calling растут с размером сильнее всего
- 3B: 6 ГБ VRAM, 30B: ~60 ГБ. Цена вопроса растёт быстрее качества
- Правило: берём самую маленькую модель, которая проходит ваш собственный оффлайн-тест

Бенчмарки: что показывает Qwen3.8-27B

Сравнение с предыдущим поколением того же размера и с одной из топовых закрытых моделей — так, как это подаёт сам вендор.



Что именно меряют

Terminal Bench

агент выполняет задачу в терминале от начала до конца

SWE-bench Pro

починить реальный баг в репозитории, проверка тестами

LiveCodeBench

свежие соревновательные задачи, защита от утечки в трейн

IFBench

буквальное следование требованиям формата

GPQA Diamond

вопросы уровня PhD: знания плюс рассуждение

HLE

намеренно очень трудные вопросы — потолок моделей

Откуда у 27B почти-SOTA

1

Гибридное внимание

Бюджет вычислений уходит не на длину, а на глубину и качество. 262k контекста доступны в реальных агентных прогонах, а не только на бумаге.

2

Вычисления на инференсе

Thinking включён по умолчанию, reasoning_effort до xhigh, рекомендованный бюджет рассуждений — 262k токенов. Заметная часть прироста живёт не в весах, а в сгенерированных токенах.

3

Пост-тренировка под агентов

Обучение и замеры на живых харнессах вроде Claude Code, а не на статических задачах. Отсюда и отрыв именно в SWE и агентных метриках.

4

Сохранение рассуждений

preserve_thinking переносит размышления прошлых ходов дальше по диалогу: меньше повторного вывода и лучше утилизация KV-кэша в многошаговых задачах.

5

МТП и большой словарь

Multi-token prediction удешевляет декодирование, а словарь на 248k экономит токены на коде и не-английских языках.

Где всё ещё отстаёт: GPQA Diamond 89.2 против 91.3 и HLE 30.8 против 40.0. Чистые знания и самые тяжёлые рассуждения по-прежнему коррелируют с размером модели.

Как читать чужие бенчмарки

Таблица в карточке модели — это маркетинговый материал, пусть и честно оформленный. Четыре вещи, которые стоит держать в голове.



Цифры от вендора

Все замеры сделала сама команда Qwen. Это не подлог, но и не независимая проверка — конкурентов настраивал не их автор.



Часть наборов внутренние

QwenSWEBench, CoWorkBench, RecreationBench — собственные бенчмарки Qwen. Именно там отрывы самые заметные: 79.0 против 63.8.



Условия очень специфичны

Общий харнесс Claude Code, temperature 1.0, окно 256k, усреднение по трём прогонам, таймаут до восьми часов на задачу. Ваш прод выглядит иначе.



Судья — тоже модель

HLE судит GPT-4o, Vision2Web — GPT-5.4. Оценка наследует ошибки судьи. И ни один столбец не показывает, сколько токенов рассуждений ушло на результат.

Что можно утверждать безопасно

Открытая модель на 27B попадает в один класс с топовыми закрытыми на задачах этого типа. Всё остальное проверяется только своим оффлайн-тестом на своих данных — и это тот же вывод, что и для Granite.

ЧАСТЬ 2

Как это развернуть

Один и тот же файл весов, четыре очень разных инженерных сценария

Четыре способа запустить одну модель



Transformers

Python-скрипт, `model.generate()`

Прототип, эксперименты, оффлайн-обработка. В прод не идёт.



vLLM / SGLang

HTTP-сервер, OpenAI API

Продовый онлайн-инференс на GPU. Батчинг, PagedAttention.



Llama.cpp / Ollama

GGUF на CPU или Mac

Локально у разработчика, edge, дешёвый пилот без GPU.



Managed API

Внешний провайдер

Быстрый старт без инфраструктуры, но данные уходят наружу.

A. Transformers — быстрый прототип

```
pip install torch accelerate transformers

from transformers import AutoModelForCausalLM, AutoTokenizer

model_path = "ibm-granite/granite-4.1-3b"
tok = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForCausalLM.from_pretrained(
    model_path, device_map="cuda") # убрать для CPU

inputs = tok(tok.apply_chat_template(chat, tokenize=False,
    add_generation_prompt=True), return_tensors="pt")
out = model.generate(**inputs, max_new_tokens=100)
```

Когда подходит

- Проверить гипотезу за 10 минут
- Батч-обработка датасета оффлайн
- Обучение и LoRA-адаптеры

Почему не прод

- Один запрос за раз, GPU простаивает
- Нет continuous batching и кэша
- Нет HTTP-слоя, метрик, лимитов

Типичная ошибка

Обернуть этот скрипт во FastAPI и выкатить как сервис. Работает на демо и разваливается на пятом одновременном пользователе: пропускная способность падает в разы, память течёт, таймауты. Если появился HTTP — переходите к варианту B.

В. vLLM и SGLang — продювый инференс

Два зрелых движка с одинаковым интерфейсом наружу: OpenAI-совместимый REST. Меняются одной переменной окружения.

vLLM

```
pip install vllm
vllm serve "ibm-granite/granite-4.1-3b"
```

SGLang

```
python3 -m sglang.launch_server \
  --model-path "ibm-granite/granite-4.1-3b" \
  --host 0.0.0.0 --port 30000
```

Клиент один и тот же — обычный OpenAI SDK или curl

```
curl -X POST http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" --data '{...}'
```

Разница в производительности с наивным вариантом А — десятки раз по токенам в секунду на том же железе.

За что мы платим этим слоем

- Continuous batching — запросы подмешиваются в батч на лету, GPU не простаивает
- PagedAttention — KV-кэш страницами, без фрагментации и лишнего резерва
- Prefix caching — общий системный промпт считается один раз
- Метрики Prometheus, стриминг, лимиты параллелизма

С. Локальный запуск: GGUF без GPU

Llama.cpp и надстройки над ним (Ollama, LM Studio, Jan) читают формат GGUF и уверенно работают на CPU и Apple Silicon.

Ollama

```
ollama run granite4.1:3b
```

Один бинарь, свой реестр моделей, локальный HTTP-API. Дефолт для разработчика.

Docker Model Runner

```
docker model run hf.co/ibm-granite/granite-4.1-3b
```

Модель как контейнер — удобно, если инфраструктура уже на Docker.

LM Studio / Jan

GUI

Для не-инженеров: интерфейс чата, выбор квантизации мышкой.

Что получаем

- Данные не покидают машину
- Нулевая стоимость запроса
- Работает офлайн, в закрытом контуре
- 3B в Q4 — около 2 ГБ на диске

Чем платим

- Единицы токенов в секунду на CPU
- Плохо масштабируется на много пользователей
- Качество ниже из-за квантизации

Сравнение подходов

Подход	Железо	Пропускная способность	Приватность	Сложность эксплуатации	Когда брать
Transformers	GPU или CPU	Низкая	Полная	Низкая	Прототип, оффлайн-батчи
vLLM / SGLang	GPU 12–24 ГБ	Высокая	Полная	Средняя	Онлайн-сервис в проде
llama.cpp / Ollama	CPU, Mac	Низкая	Полная	Очень низкая	Локально, edge, пилот
Docker / K8s + vLLM	Пул GPU	Высокая	Полная	Высокая	Масштаб, несколько команд
Managed API	Не нужно	Высокая	Ограниченная	Минимальная	Скорость важнее контроля

Практичный маршрут: прототип на Transformers → локальная проверка через Ollama → пилот на vLLM на одной карте → Kubernetes, когда появится вторая команда-потребитель.

Чего не хватает между демо и продам



Оценка

Свой набор из 100–300 примеров с эталонами. Без него любое обновление модели — прыжок в темноту.



Наблюдаемость

Логи промптов и ответов, задержка до первого токена, токены в секунду, стоимость на запрос.



Версионирование

Фиксируем ревизию весов и версию движка. Ответы модели не воспроизводятся между версиями.



Безопасность

Фильтрация входа и выхода. У IBM для этого отдельная модель Granite Guardian — классификатор рисков.



Лимиты и деградация

Rate limit, таймауты, очередь, запасной путь при недоступности GPU.



Лицензия и данные

Apache 2.0 у Granite снимает вопросы к использованию, но не к данным, которые вы в модель отправляете.

ЧТО ЗАБРАТЬ С СОБОЙ

Пять тезисов

- 1 LLM — это цикл предсказания токена. Всё остальное мы строим вокруг него сами.
- 2 Планируйте память по формуле «веса + KV-кэш», а не по числу параметров.
- 3 Маленькая открытая модель на 3B закрывает больше задач, чем кажется — проверяйте на своих данных.
- 4 Способ развёртывания выбирается по нагрузке и приватности, а не по моде.
- 5 Разница между демо и продом — не в модели, а в оценке, лимитах и наблюдаемости.

Вопросы?